

SimpleRepeatSeq: clasificación de secuencias de ADN de repetición simple mediante codificación por k -meros y aprendizaje automático

Cristian Fontanilla¹, Mateo Aguirre¹, Santiago García¹, and Leandro Panesso¹

¹Universidad de Caldas — Maestría en Ingeniería Computacional, Bioinformática Computacional

Resumen

Motivación: Los *simple repeats* (motivos de 1–6 bases repetidos en tándem) son abundantes en los genomas y su detección es crítica en ensamblaje, alineamiento de lecturas y diagnóstico clínico (expansiones de CAG/Huntington, CGG/X-Frágil, GGGGCC/ELA) y forense (STRs). Las herramientas clásicas (TRF, RepeatMasker) se basan en reglas y alineamiento; falta una comparación sistemática de codificaciones numéricas y modelos de aprendizaje automático para esta tarea de clasificación.

Resultados: Sobre un conjunto balanceado de 888 secuencias evaluamos exhaustivamente 6 codificaciones $\times$ 3 preprocesamientos $\times$ 8 modelos mediante validación cruzada estratificada 10-fold. Las codificaciones por frecuencia de k -meros superan claramente a las posicionales; la configuración **Random Forest** + **k -meros** ($k=4$) + **estandarización** alcanza Exactitud = AUC = $F1_w = 1.000$, y se distribuye como modelo final. k -meros $k=3$ logra el mismo rendimiento con solo 125 características.

Disponibilidad: Código, notebooks ejecutables, figuras y modelo en <https://github.com/leandropa00/SimpleRepeatSeq>.

1 Introducción

Los *simple sequence repeats* (microsatélites, STRs y satélites) constituyen una fracción sustancial del genoma humano y de muchos organismos, y su correcta identificación es relevante para el ensamblaje *de novo*, el alineamiento de lecturas cortas y el diagnóstico molecular: las expansiones de tripletes CAG, CGG y CTG causan, respectivamente, enfermedad de Huntington, síndrome de X-Frágil y distrofia miotónica, mientras que la expansión hexanucleotídica GGGGCC se asocia a la esclerosis lateral amiotrófica; los STRs tetranucleotídicos son la base de la identificación forense de ADN [5, 1].

Las herramientas de referencia —Tandem Repeats Finder (TRF) [1] y RepeatMasker [2]— detectan repeticiones mediante modelos de alineamiento y bibliotecas de consenso, pero no abordan el problema como una tarea de *clasificación supervisada* ni comparan representaciones numéricas de la secuencia. Las aproximaciones libres de alineamiento basadas en la composición de k -meros han demostrado ser eficaces para comparar y clasificar secuencias [3, 4]. Aquí cerramos esa brecha evaluando **sistemáticamente** qué codificación y qué modelo separan mejor secuencias de repetición simple de secuencias genómicas normales, y publicamos un clasificador reproducible junto con una herramienta de línea de comandos.

2 Métodos

2.1 Conjunto de datos

Se emplearon 888 secuencias balanceadas (444 *simpleSeq* + 444 *no_simpleSeq*), de 100–500 pb y con 4–12 % de ruido de secuenciación simulado. La clase *simpleSeq* incluye telómeros (TTAGGG), expansiones patológicas (CAG, CGG, CTG, GAA, GGGGCC), STRs forenses (HUMTH01, vWA, D3S1358), satélites cen-

troméricos y microsatélites di/trinucleotídicos; la clase *no_simpleSeq* reúne exones, intrones, promotores CpG, 5'/3'UTR, enhancers, secuencia mitocondrial y SINEs, con composición nucleotídica calibrada a partir de ENCODE/Ensembl.

2.2 Codificaciones y preprocesamiento

Se implementaron seis codificaciones: frecuencias normalizadas de k -meros con $k = 3, 4, 6$ (125, 625 y 15 625 características; alfabeto ACGTN), y tres codificaciones posicionales por base —DAX (A=2, C=0, G=3, T=1), EIIP (energía de interacción electrón-ión) y complementaria (escala purina/pirimidina)— con relleno a longitud 500. Cada codificación se evaluó con tres estrategias de preprocesamiento: crudo, estandarización (**StandardScaler**) y PCA reteniendo el 96 % de la varianza.

2.3 Modelos y validación

Se entrenaron ocho clasificadores: k -NN, SVM (núcleo RBF), regresión logística, LDA, Naive Bayes gaussiano, Random Forest [6], XGBoost [7] y una red neuronal totalmente conectada (DNN). Las 144 combinaciones codificación $\times$ preprocesamiento $\times$ modelo se evaluaron con validación cruzada estratificada 10-fold, reportando exactitud, $F1$ ponderado, precisión, recall y AUC. La canalización completa se implementó con scikit-learn [8], Biopython [9], XGBoost [7] y Keras/TensorFlow [10].

3 Resultados

La codificación de la secuencia domina el rendimiento. Las representaciones por k -meros son casi perfectamente separables: con $k = 3$ y $k = 4$, múltiples modelos alcanzan exactitud y AUC iguales a 1.000 en validación cruzada (Tabla 1, Fig. 1). Las codificaciones posicionales rinden por debajo y son mucho más sensibles al

Cuadro 1. Mejor configuración por codificación (CV 10-fold, $n = 888$). T(s): tiempo medio por *fold*; est. = estandarizado, cru. = crudo.

Codif.	Mod./Prep.	Exact.	F1 _w	AUC	T(s)	N.car.
<i>k</i> -mer4	RF / est.	1.000	1.000	1.000	0.52	625
<i>k</i> -mer3	RF / cru.	1.000	1.000	1.000	0.46	125
<i>k</i> -mer6	SVM / PCA	1.000	1.000	1.000	5.05	15 625
compl.	SVM / cru.	0.989	0.989	1.000	0.25	500
dax	XGB / cru.	0.987	0.987	0.999	0.14	500
eiip	NB / PCA	0.985	0.985	1.000	0.07	500

modelo: la peor combinación (complementaria + crudo + *k*-NN) cae al azar (AUC = 0.50), lo que confirma que la elección de la representación, y no solo del clasificador, es determinante.

Ordenando por AUC medio, el ranking de codificaciones es $k\text{-mer4} > k\text{-mer3} > k\text{-mer6} > \text{eiip} > \text{dax} > \text{complementaria}$, y el de modelos RF > XGB > SVM > LR > LDA > NB > *k*-NN > DNN. La tabla completa con las 144 combinaciones (exactitud, F1, precisión, recall, AUC, desviaciones, tiempo y características) está en el repositorio ([data/encoded/resultados_experimento.csv](#)).

El mapa de calor comparativo (Fig. 1) resume las tres métricas para los ocho modelos a lo largo de las seis codificaciones y los tres preprocesamientos: el rendimiento máximo se concentra en los bloques de *k*-meros. Las curvas ROC del mejor modelo de cada codificación (Fig. 2) son prácticamente ideales, con AUC = 1.000 para todas las representaciones por *k*-meros. La matriz de confusión del mejor modelo (Fig. 3) muestra separación perfecta (444/444 en ambas clases, sin falsos positivos ni negativos); la Fig. 4 contrasta el patrón de errores de los ocho modelos sobre la mejor codificación.

Por parsimonia y velocidad seleccionamos como modelo desplegado **Random Forest + *k*-mer4 + estandarización** (625 características, < 0.6 s por *fold*), almacenado en `models/best_model.pkl`. *k*-mer6, pese a igualar el rendimiento, multiplica por ~25 el número de características y el coste computacional (Fig. 5: SVM hasta ~58 s por *fold*), por lo que no se justifica frente a *k*-mer3/4.

4 Discusión

La separabilidad casi perfecta de las codificaciones por *k*-meros es coherente con la naturaleza del problema: un *simple repeat* se define por la *periodicidad composicional* de su secuencia, que la frecuencia de *k*-meros captura de forma directa, mientras que las codificaciones posicionales diluyen esa señal. Que un modelo tan simple como *k*-NN sobre *k*-mer3 alcance el 100 % sugiere que, en este conjunto, las clases son linealmente separables en el espacio composicional.

Esto constituye también la principal **limitación**: el conjunto es semi-sintético (composición calibrada con datos reales pero secuencias generadas), por lo que los valores perfectos no deben extrapolarse a datos genómicos reales sin validación adicional, donde se esperan fronteras de decisión más difusas (repeticiones de generadas, transiciones repeat/no-repeat, sesgos de secuenciación). Como trabajo futuro proponemos validar

sobre regiones anotadas reales, incorporar repeticiones imperfectas y comparar contra TRF/RepeatMasker en datos genómicos. Para uso práctico recomendamos la configuración parsimoniosa (*k*-mer3/4 con un modelo de árboles), que combina exactitud máxima, bajo coste y características interpretables.

Disponibilidad y requisitos

Repositorio: <https://github.com/leandropa00/SimpleRepeatSeq>. **Lenguaje:** Python ≥ 3.9. **Dependencias:** biopython, numpy, pandas, matplotlib, seaborn, scikit-learn, xgboost, tensorflow ≥ 2.13, scikeras, joblib. **Instalación:** `pip install -r requirements.txt`. **Uso:** ejecutar `notebooks/01_load_data.ipynb` y `02_experiments.ipynb`, o clasificar un FASTA con `python classify.py -fasta entrada.fasta -model models/best_model.pkl -encoding kmer4`.

Referencias

- [1] Benson G. Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Res.* 1999;27(2):573–580.
- [2] Smit AFA, Hubley R, Green P. RepeatMasker Open-4.0. 2013–2015. Disponible en: <http://www.repeatmasker.org>.
- [3] Vinga S, Almeida J. Alignment-free sequence comparison—a review. *Bioinformatics.* 2003;19(4):513–523.
- [4] Wood DE, Salzberg SL. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biol.* 2014;15(3):R46.
- [5] Gymrek M, Golan D, Rosset S, Erlich Y. lobSTR: a short tandem repeat profiler for personal genomes. *Genome Res.* 2012;22(6):1154–1162.
- [6] Breiman L. Random forests. *Mach Learn.* 2001;45(1):5–32.
- [7] Chen T, Guestrin C. XGBoost: a scalable tree boosting system. En: *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.* 2016. p. 785–794.
- [8] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: machine learning in Python. *J Mach Learn Res.* 2011;12:2825–2830.
- [9] Cock PJA, Antao T, Chang JT, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics.* 2009;25(11):1422–1423.
- [10] Abadi M, Barham P, Chen J, et al. TensorFlow: a system for large-scale machine learning. En: *12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*. 2016. p. 265–283.

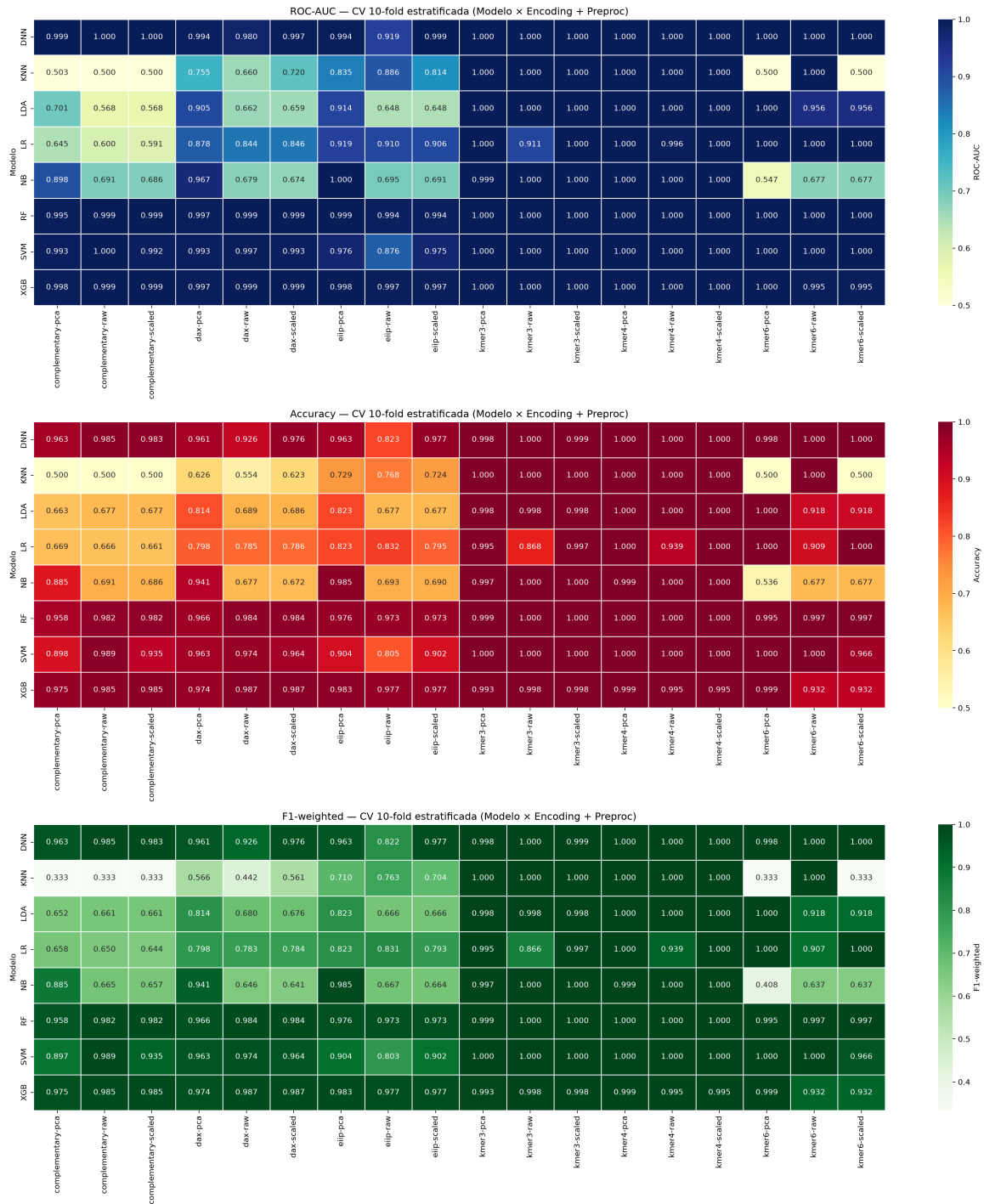


Figura 1. Mapa de calor comparativo (AUC, exactitud y F1 ponderado) de los ocho modelos a lo largo de las seis codificaciones y los tres preprocesamientos, bajo validación cruzada estratificada 10-fold. Las codificaciones por k -meros (bloques de la derecha) concentran el rendimiento máximo.

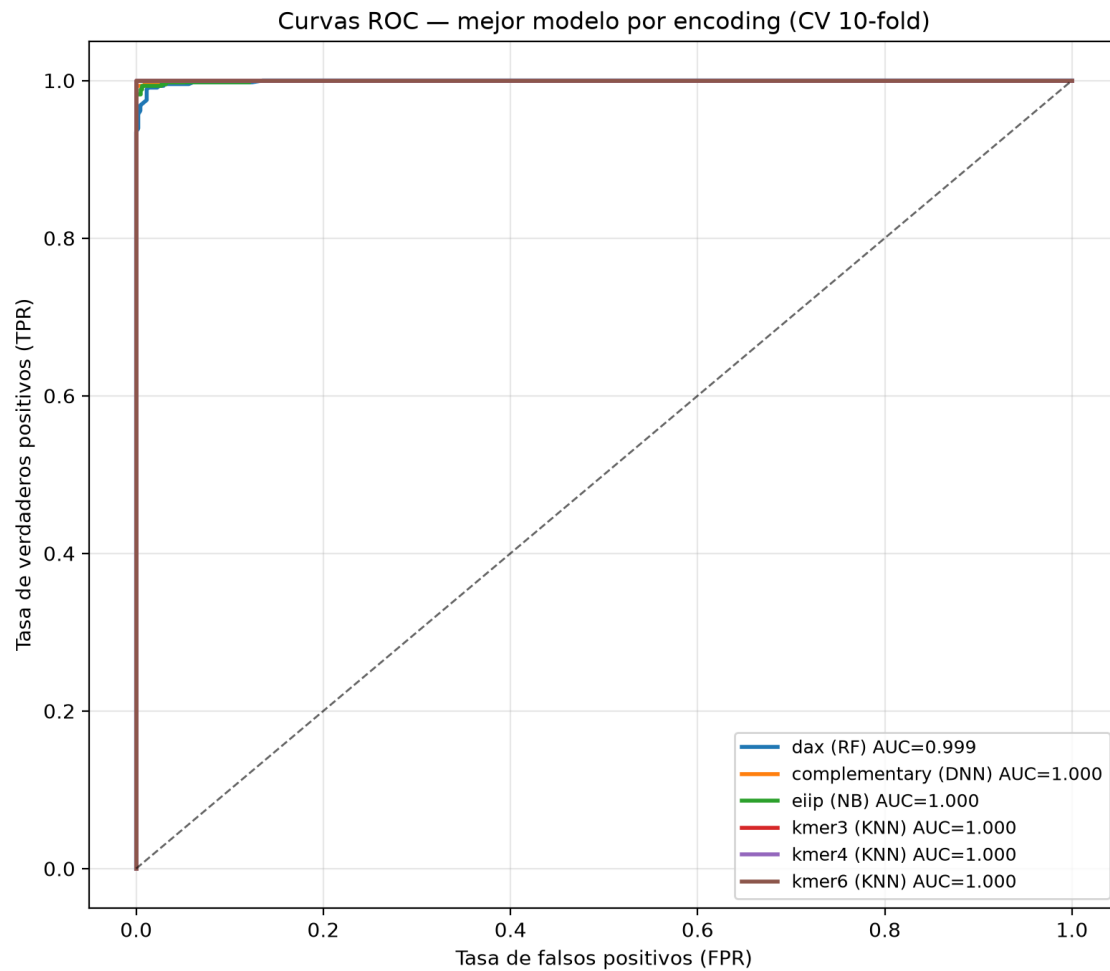


Figura 2. Curvas ROC del mejor modelo por codificación (CV 10-fold). Todas las representaciones por k -meros alcanzan $AUC = 1.000$.

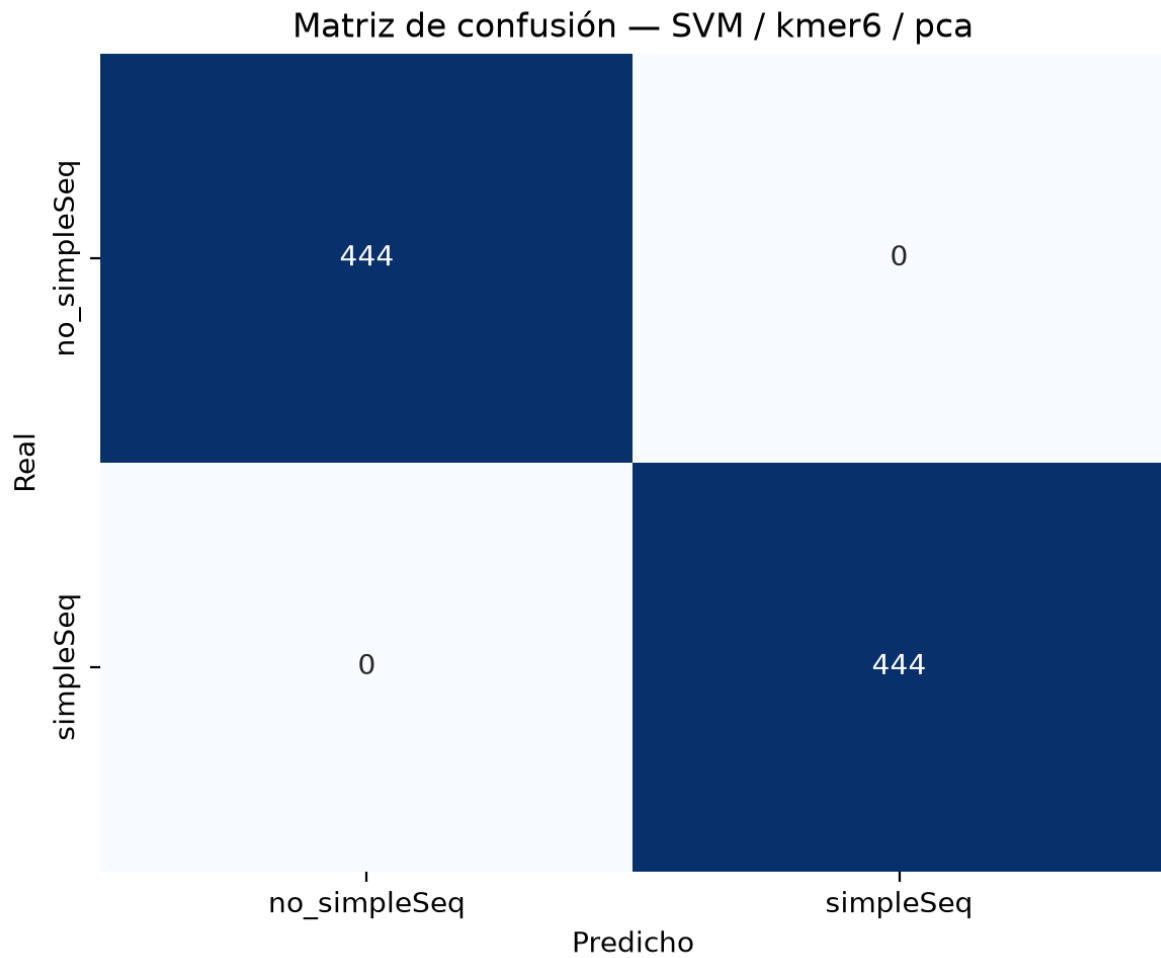


Figura 3. Matriz de confusión de una configuración con rendimiento perfecto: 444/444 secuencias correctamente clasificadas en ambas clases. El modelo desplegado (RF / *k*-mer4 / estandarización) produce la misma matriz con una fracción del coste.

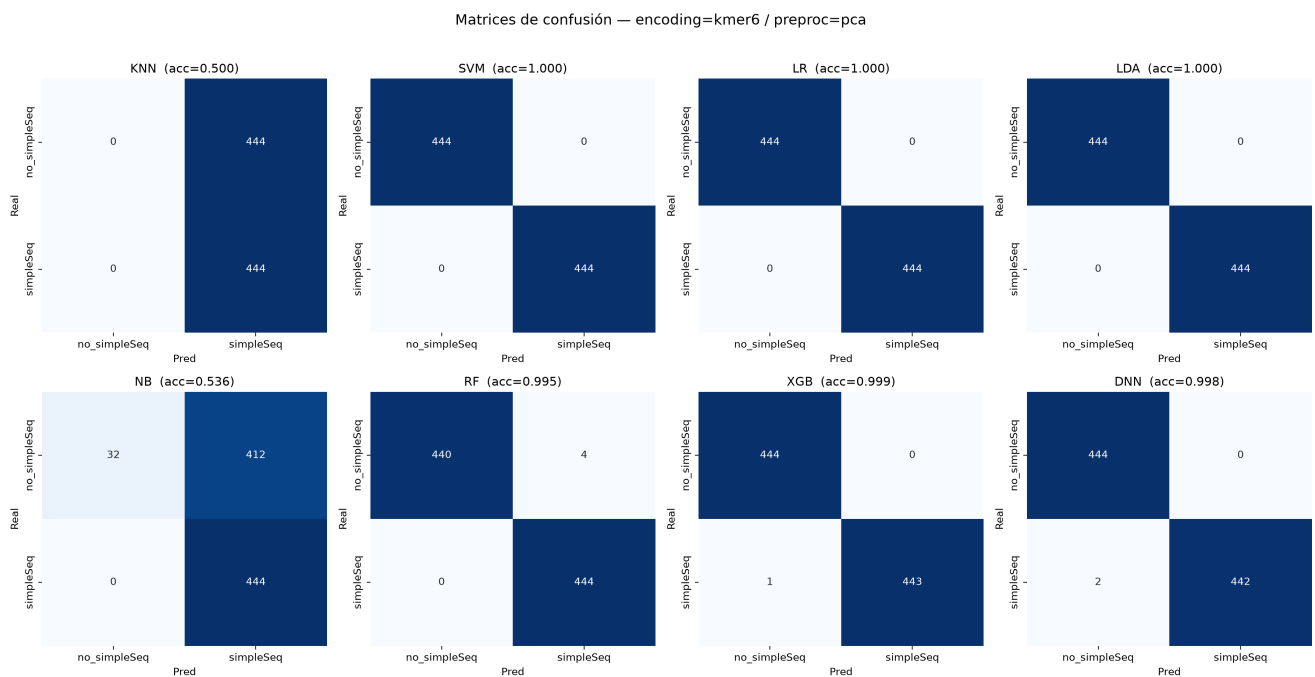


Figura 4. Matrices de confusión de los ocho modelos sobre la mejor codificación, mostrando el patrón de errores de cada clasificador.

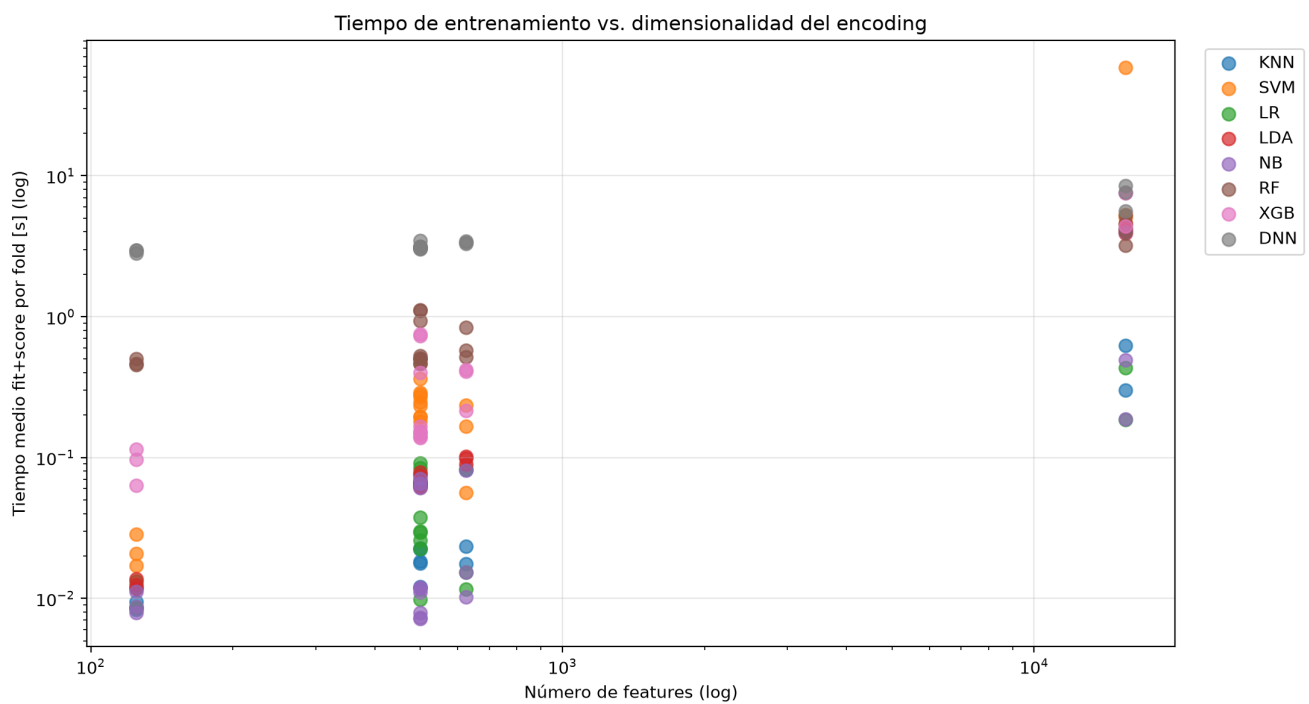


Figura 5. Tiempo medio de entrenamiento por *fold* frente a la dimensionalidad de la codificación (ejes logarítmicos). El coste crece con el número de características; SVM sobre *k*-mer6 (15 625 características) es el caso más costoso.